

Estratégias de Teste de Software

Parte 1

O Teste de Software: Uma Visão Destrutiva

"O teste é o processo de execução de um programa com a intenção de encontrar erros."

— Roger S. Pressman

- **Objetivo Primário:** Encontrar o maior número de erros com o mínimo de esforço e tempo.
- **Teste Bem-Sucedido:** Aquele que revela um erro que ainda não havia sido descoberto.
- **Teste não é Depuração:** Teste revela a presença do erro; Depuração localiza e corrige a causa.

Fundamentos: Verificação vs. Validação (V&V)

Aspecto	Verificação (V)	Validação (V)
Pergunta-chave	"Estamos construindo o produto corretamente ?"	"Estamos construindo o produto certo ?"
Foco	Conformidade com as especificações (Requisitos, Design, Código)	Atendimento às expectativas do cliente (Necessidades reais)
Atividades	Revisões, Inspeções, Análise de Caminho, Testes de Caixa Branca	Teste de Aceitação, Teste de Sistema, Teste de Caixa Preta
Quando	Durante o desenvolvimento (Fase de construção)	Após o desenvolvimento (Fase de teste e aceitação)

Os Sete Princípios do Teste (Parte 1)

1

Teste demonstra a presença de defeitos

O teste não prova a ausência de erros. Um teste bem-sucedido é aquele que revela um defeito que ainda não havia sido descoberto. Portanto, o objetivo é encontrar erros, não provar que o software está correto.

2

Teste exaustivo é impossível

O número de caminhos de execução em um programa é potencialmente infinito. Testar todas as combinações de entrada é impraticável. Portanto, o testador deve ser estratégico e focar nos testes mais prováveis de encontrar erros.

3

Teste antecipado economiza tempo e dinheiro

Defeitos descobertos no início do ciclo de desenvolvimento são muito mais baratos de corrigir do que aqueles descobertos tarde. Esta é a base do conceito "Shift-Left" na Engenharia de Software moderna, onde o teste é integrado desde as fases iniciais.

Os Sete Princípios do Teste (Parte 2)

4 Agrupamento de Defeitos

A maioria dos defeitos está concentrada em um pequeno número de módulos. O testador deve focar seus esforços nos módulos de maior risco.

5 Paradoxo do Pesticida

Os mesmos testes param de encontrar novos defeitos após várias execuções. É necessário criar novos casos de teste e atualizar continuamente a suíte de testes.

6 Teste Depende do Contexto

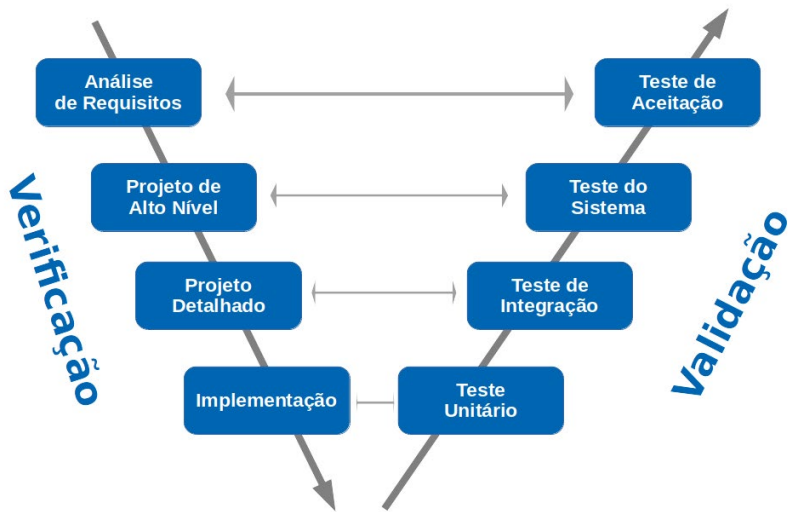
A estratégia de teste deve ser adaptada ao contexto específico. Testar um sistema de segurança é diferente de testar um jogo ou um sistema embarcado.

7 A Falácia da Ausência de Erros

Software 99% livre de erros, mas que não atende às necessidades do cliente, é inútil. O teste deve validar que o software atende aos requisitos e às expectativas.

Estratégia Geral: O Modelo em 'V' do Teste

O Modelo V correlaciona cada fase de desenvolvimento com um nível correspondente de teste. O teste não é uma fase final, mas uma atividade paralela que acompanha o desenvolvimento.



Princípio-chave: O Teste de Unidade valida o Design de Componentes; o Teste de Integração valida o Design de Sistema; o Teste de Sistema valida os Requisitos. Cada nível de teste corresponde a uma fase de desenvolvimento.

Organização para o Teste: A Equipe SQA

O que é SQA?

- **SQA (Software Quality Assurance):** Grupo independente responsável por planejar, supervisionar e executar o teste de software.
- **Função Principal:** Garantir que o processo de teste seja rigoroso e que o software atenda aos padrões de qualidade definidos.

Por que a Independência é Crucial?

Princípio Fundamental: O testador não deve ser o desenvolvedor do código testado. A independência garante objetividade e uma mentalidade destrutiva.

- **Maior Objetividade:** O desenvolvedor tende a testar apenas os caminhos "felizes" (happy paths). O testador independente busca quebrar o software.
- **Maior Probabilidade de Encontrar Erros:** A mentalidade destrutiva do testador independente leva a testes mais rigorosos e abrangentes.
- **Advogado do Cliente:** A equipe SQA atua como representante dos interesses do cliente dentro da organização de desenvolvimento.

Tipos de Teste: Caixa Branca vs. Caixa Preta

Aspecto	Caixa Branca (Estrutural)	Caixa Preta (Comportamental)
Foco	Estrutura interna do software (código-fonte, lógica, caminhos de execução).	Funcionalidade e comportamento externo (requisitos, interface, entrada/saída).
Objetivo	Garantir que todos os caminhos lógicos, decisões e condições sejam exercitados.	Garantir que as funções atendem aos requisitos sem conhecer a implementação interna.
Exemplos	Teste de Caminho Básico, Cobertura de Linhas, Cobertura de Condições, Teste de Loop.	Teste de Equivalência, Teste de Valor Limite, Teste de Transição de Estado.
Responsabilidade	Primária do desenvolvedor (Teste de Unidade).	Primária da equipe SQA (Testes de Integração e Sistema).

Teste de Unidade (Unit Testing): O Foco

- **O que é**

Teste do menor elemento de software: um módulo, um componente, uma classe ou uma função. A unidade é testada isoladamente, sem dependências externas.

- **Objetivo**

Verificar a lógica interna da unidade, garantindo que ela funciona conforme o design. Testar a interface local, as estruturas de dados internas e os caminhos de execução.

- **Foco**

Garantir que o módulo, isoladamente, funciona corretamente. O teste de unidade é responsabilidade primária do desenvolvedor, realizado durante a fase de codificação.

- **Limitações**

O teste de unidade não detecta erros de interface entre módulos, erros de integração, ou erros que dependem de recursos externos (banco de dados, rede, sistema de arquivos).

Teste de Unidade: Estrutura do Teste

1

Interface do Módulo

Testar o fluxo de dados de entrada e saída. Garantir que o módulo recebe os dados corretos, processa-os adequadamente e retorna os resultados esperados.

2

Estruturas de Dados Locais

Garantir a integridade dos dados internos do módulo. Verificar que as estruturas de dados (variáveis, arrays, registros) mantêm valores válidos durante a execução.

3

Caminhos de Execução Independentes

Testar todos os caminhos lógicos do módulo (Teste de Caixa Branca). Usar técnicas como Teste de Caminho Básico e Cobertura de Condição para garantir que todas as decisões são exercitadas.

4

Tratamento de Erros

Testar o comportamento do módulo em condições de erro. Exemplos: divisão por zero, entrada inválida, overflow, acesso a arquivo inexistente. O módulo deve falhar de forma controlada.

O Ambiente de Teste de Unidade: Drivers e Stubs

Driver

Programa que **simula o módulo que chama** o módulo em teste. O Driver fornece os dados de entrada (test cases) e captura os resultados de saída.

Quando usar:

Integração Bottom-Up, quando o módulo em teste chama outros módulos que ainda não foram implementados.

Stub

Módulo '**dummy**' que **simula a interface** de módulos chamados pelo módulo em teste. O Stub retorna valores predefinidos para simular o comportamento de módulos não disponíveis.

Quando usar:

Integração Top-Down, quando o módulo em teste depende de outros módulos que ainda não foram implementados.

Propósito: Isolamento do Módulo em Teste

O objetivo principal de Drivers e Stubs é **isolar o módulo em teste**, removendo todas as dependências externas. Isso permite que o testador:

- Teste o módulo de forma independente, sem esperar por outros módulos.

- Controle completamente as entradas (via Driver) e as saídas simuladas (via Stub).

- Identifique erros no módulo em teste, sem confundi-los com erros em dependências externas.

- Simule condições de erro ou casos extremos que seriam difíceis de reproduzir com módulos reais.

Teste de Unidade em Software Orientado a Objeto (OO)

Mudanças em Relação ao Teste de Unidade Funcional

- **Unidade de Teste:** A Classe (e não o módulo funcional). Cada classe é testada como uma unidade, com todos os seus métodos e atributos.
 - **Teste de Métodos e Estado:** Além de testar cada operação (método) da classe, é necessário testar o estado do objeto após a execução das operações.
 - **Sequência de Operações:** O teste deve considerar a sequência de chamadas de métodos, garantindo que o objeto não entra em um estado inválido.
-

Desafios Específicos de OO

- **Herança:** Testes de superclasses devem ser re-executados para subclasses, considerando o comportamento herdado e o comportamento sobrescrito.
- **Polimorfismo:** O comportamento do método depende do tipo de objeto em tempo de execução. O teste deve considerar diferentes tipos polimórficos.
- **Encapsulamento:** A ocultação de informações dificulta o teste de caixa branca, pois o acesso a estados internos é restringido. O teste deve focar na interface pública.

Teste de Integração: Construindo a Estrutura

- **Objetivo Principal**

Montar o programa de forma sistemática, garantindo que os módulos interajam e se comuniquem corretamente. O foco muda do funcionamento isolado de cada módulo para o funcionamento integrado do sistema.

- **Foco Principal**

Erros de interface entre módulos, incluindo: fluxo de dados incorreto, comunicação de controle inadequada, dados corrompidos durante a transferência, e incompatibilidade de tipos de dados entre módulos chamadores e chamados.

- **Desafio Crítico**

O erro de interface é o **mais difícil de detectar e corrigir**, pois envolve múltiplos módulos e a causa pode estar em qualquer um deles. Portanto, o teste de integração deve ser realizado de forma sistemática e controlada, não aleatória.

Abordagens de Integração

Aspecto	Big Bang (Não Recomendado)	Integração Incremental (Recomendada)
Abordagem	Todos os módulos são combinados de uma vez, após o término de todos os testes de unidade.	Módulos são integrados um a um ou em pequenos grupos, de forma sistemática e controlada.
Problemas	Localização de erros extremamente difícil (sistema funciona como uma "caixa preta" gigante). Pânico e atrasos na fase de teste.	Localização de erros mais fácil. Teste contínuo e feedback imediato. Detecção precoce de erros de interface.
Impacto	Alto risco. Erros descobertos tardiamente são caros de corrigir. Pode resultar em atrasos significativos no cronograma.	Menor risco. Erros são corrigidos imediatamente. Cronograma mais previsível e controlado.

Integração Top-Down (Incremental)

Estratégia

- Começa pelo **módulo de controle principal** (topo da hierarquia) e desce progressivamente.
- Módulos **inferiores são substituídos por Stubs** (módulos 'dummy' que simulam seu comportamento).
- À medida que cada nível é testado, seus Stubs são substituídos pelos módulos reais.

Vantagens

Detecção Precoce de Erros de Interface de Alto Nível: Problemas no fluxo de controle principal são descobertos rapidamente.

- Uma **versão de trabalho é obtida mais cedo**, permitindo demonstrações e feedback inicial.
- Alinha-se bem com a **arquitetura hierárquica** de muitos sistemas.

Desvantagens

- A **criação de Stubs pode ser complexa**, especialmente para módulos com comportamento complicado.
- **Testes de módulos inferiores são adiados**, deixando erros de baixo nível para serem descobertos tardiamente.
- Difícil testar **operações críticas de baixo nível** (e.g., I/O, manipulação de dados) isoladamente.

Integração Bottom-Up (Incremental)

Estratégia

- Começa pelos **módulos atômicos (folhas)** da hierarquia, aqueles que não chamam outros módulos.
- Módulos **superiores são substituídos por Drivers** (programas de teste que simulam o comportamento dos chamadores).
- À medida que cada nível é testado, seus Drivers são substituídos pelos módulos reais, subindo progressivamente na hierarquia.

Vantagens

Módulos de Baixo Nível Completamente Testados: Operações críticas (I/O, manipulação de dados, acesso a banco de dados) são testadas isoladamente e completamente.

- Drivers são **geralmente mais simples de criar** do que Stubs, pois simulam apenas a entrada de dados.
- **Menos risco de erros não detectados** em operações críticas de baixo nível.

Desvantagens

- O **fluxo de controle principal é testado apenas no final**, deixando erros de interface de alto nível para serem descobertos tardiamente.
- **Nenhuma versão de trabalho é obtida cedo**, dificultando demonstrações e feedback inicial do cliente.
- Requer **conhecimento detalhado da estrutura do sistema** para criar Drivers apropriados.

Integração Híbrida (Sanduíche)

O que é

- Combinação de **Top-Down** e **Bottom-Up**, integrando módulos em múltiplos níveis simultaneamente.
- A integração ocorre no **nível intermediário** da hierarquia, com Top-Down acima e Bottom-Up abaixo.

Como Funciona

Estratégia: Divide-se o sistema em três camadas: superior (integração Top-Down), intermediária (ponto de junção), e inferior (integração Bottom-Up). As equipes podem trabalhar em paralelo em diferentes partes da hierarquia.

- Módulos superiores usam **Stubs** para módulos intermediários.
- Módulos inferiores usam **Drivers** para módulos intermediários.
- Quando os testes superiores e inferiores são concluídos, os Stubs e Drivers são substituídos pelos módulos reais.

Vantagens e Recomendação

- **Trabalho Paralelo:** Diferentes equipes podem trabalhar simultaneamente em diferentes partes da hierarquia.
- **Otimização:** Minimiza a necessidade de Stubs e Drivers complexos, comparado com Top-Down ou Bottom-Up puro.

Recomendação Profissional: A integração Sanduíche é a estratégia mais comum na prática profissional, pois oferece o melhor equilíbrio entre eficiência, controle e risco.